

课程笔记

进程的地址空间；mmap, munmap, mprotect

南京大学 操作系统原理 2026 春季学期 第 06 讲

lecture-to-notes

2026 年 3 月 24 日



视频作者/频道：绿导师原谅你了（蒋炎岩 jyy）

发布日期：2026 年春季学期

视频时长：1 小时 19 分钟

视频链接：<https://www.bilibili.com/video/BV18nAjz9EcW>

目录

1 进程的地址空间	2
1.1 地址空间和指针	2
1.2 内存中每个字节的来源	3
1.3 亲手查看地址空间	4
1.4 本章小结	4
2 mmap: 内存映射	4
2.1 从 minimal.S 出发: 地址空间里什么也没有	5
2.2 mmap 的语义	5
2.3 strace 观察 mmap	6
2.4 本章小结	6
3 入侵进程的地址空间	6
3.1 Hacking Address Spaces	7
3.2 畅想的空間	8
3.3 本章小结	8
4 总结与延伸	8
4.1 讲者的核心信息	8
4.2 结构化提炼	9
4.3 跨章节关联	9
4.4 与前几讲的关联	9
4.5 开放问题	9
4.6 拓展阅读	9

1 进程的地址空间

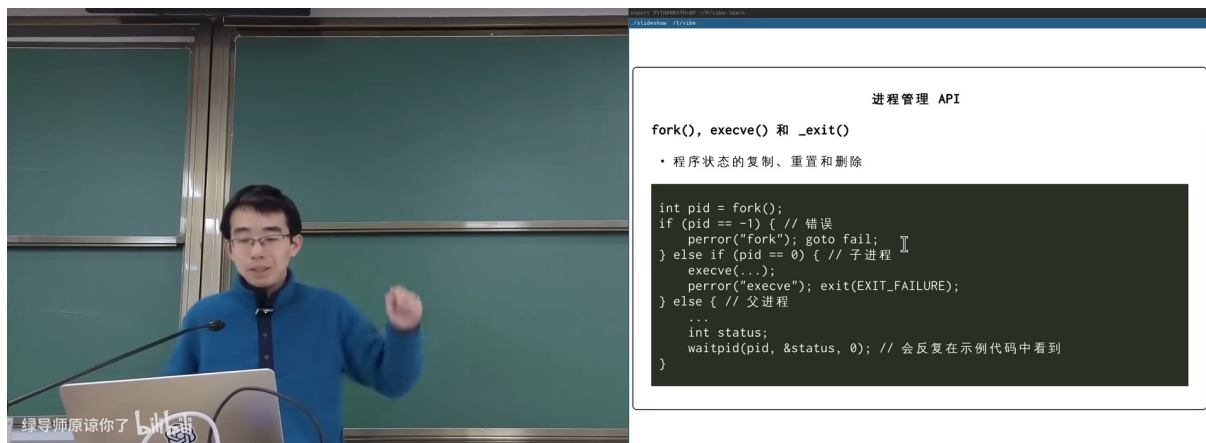


图 1: 回顾: fork(), execve(), _exit() 的代码模式¹

jyy 首先快速回顾了第 05 讲的进程管理 API——fork/execve/_exit, 然后引出新问题: 进程的内存长什么样?

1.1 地址空间和指针

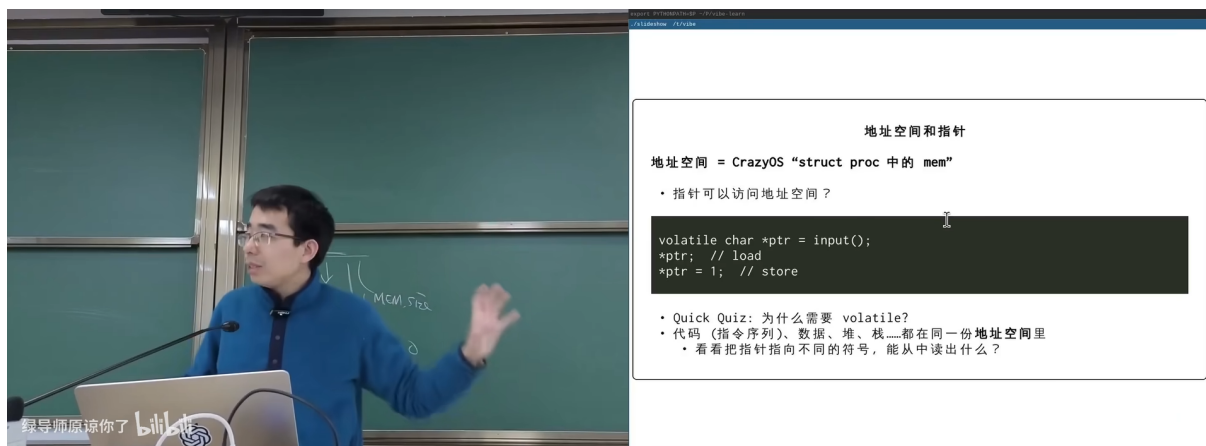


图 2: 地址空间 = CrazyOS 的 struct proc 中的 mem; 指针可以访问地址空间²

地址空间的本质

地址空间 = 进程能访问的所有内存的集合。

类比 CrazyOS: 每个 struct proc 里有一个 mem 数组——这就是那个进程的“全部世界”。

真实 OS 中, 每个进程有自己独立的地址空间, 通过 MMU (内存管理单元) 实现隔离。

指针 = 地址空间中某个位置的标签。*ptr 就是读写地址空间中的一个字节。

¹ 视频画面时间区间: 00:00:45–00:01:15。帧实际内容: slide “进程管理 API”, 展示 fork/execve/_exit 的典型用法代码 (pid = fork(); if pid == -1 错误; else if pid == 0 子进程 execve; else 父进程 waitpid)。

² 视频画面时间区间: 00:07:00–00:07:30。帧实际内容: slide “地址空间和指针”, 显示 “地址空间 = CrazyOS ‘struct proc’ 中的 mem”, 以及 volatile char *ptr = input(); *ptr // load; *ptr = 1; // store 代码。包含 Quick Quiz: 为什么需要 volatile?

```

1 volatile char *ptr = input();
2 *ptr;          // load: 从地址空间读取
3 *ptr = 1;      // store: 向地址空间写入

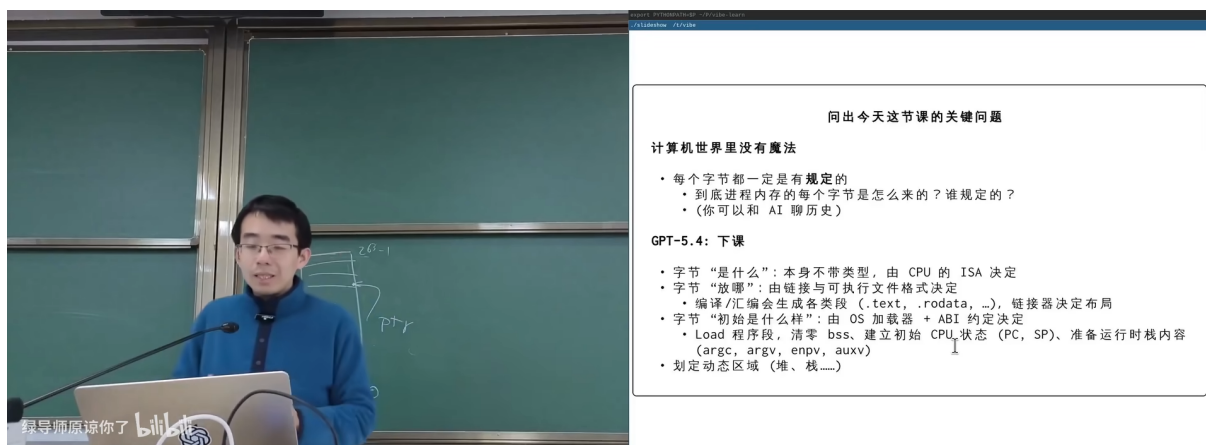
```

Listing 1: 指针访问地址空间

为什么需要 volatile?

Quick Quiz: 没有 volatile, 编译器可能优化掉对 *ptr 的访问 (认为没有副作用)。在系统编程中, 内存可能被其他进程、硬件或 OS 修改——volatile 告诉编译器“每次都要真正去读/写内存”。

1.2 内存中每个字节的来源



问出今天这节课的关键问题

计算机世界里没有魔法

- 每个字节都一定是有规定的
 - 到底进程内存的每个字节是怎么来的? 谁规定的?
 - (你可以和 AI 聊历史)

GPT-5.4: 下课

- 字节“是什么”: 本身不带类型, 由 CPU 的 ISA 决定
- 字节“放哪”: 由链接与可执行文件格式决定
 - 编译/汇编会生成各类段 (.text, .rodata, ...), 链接器决定布局
- 字节“初始是什么样”: 由 OS 加载器 + ABI 约定决定
 - Load 程序段, 清零 .bss、建立初始 CPU 状态 (PC, SP)、准备运行时栈内容 (argc, argv, envp, auxv)
- 划定动态区域 (堆、栈.....)

图 3: 今天这节课的关键问题: 内存中每个字节是怎么来的?³

jyy 提出了本讲的核心问题:

关键问题: 每个字节从哪来?

计算机世界没有魔法。进程内存中的每一个字节都有明确的来源:

1. 字节的“出生”: 由 CPU 的 ISA 决定
2. 字节的“类型”: 由 linker 决定 (.text, .rodata, ...), 编译器决定布局为具体的 section
3. 字节的“初始化”: 由 OS 的 loader + ABI 决定
4. Load 程序: 建立 .bss, 建立 CPU 状态 (PC, SP), 准备运行时数据 (argc, argv, envp)

你可以让 AI 帮你总结——但你需要能验证它说的对不对。

³视频画面时间区间: 00:14:30-00:15:00。帧实际内容: slide “今天这节课的关键问题”——计算机世界没有魔法; 到底进程内存的每个字节是怎么来的? GPT-5.4 总结: 字节“出生”由 CPU 的 ISA 决定, 类型由 linker 决定 (text, rodata, ...), 编译器决定布局。Load 程序 bss 建立 CPU 状态 (PC, SP), 准备运行时载入 (argc, argv, envp)。

1.3 亲手查看地址空间

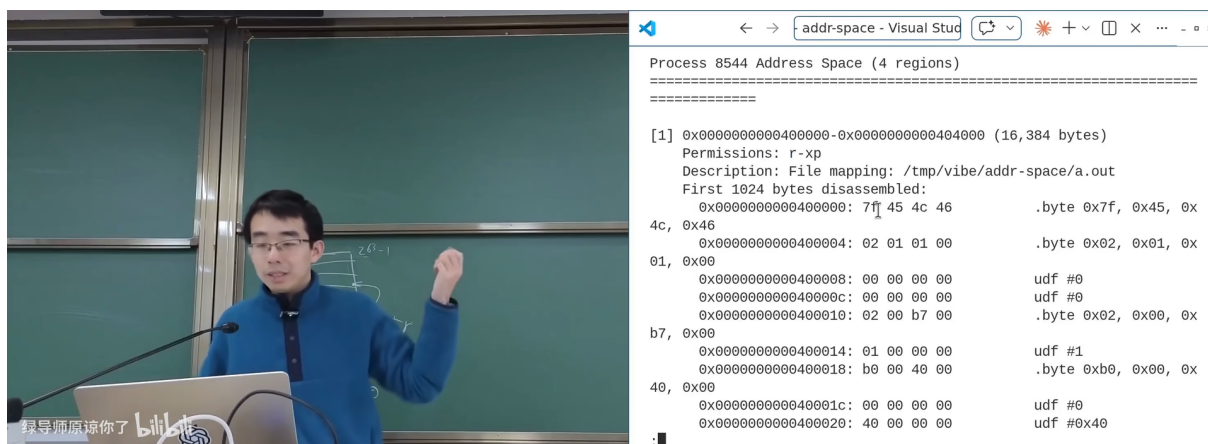


图 4: addr-space 工具: Process 8544 Address Space (4 regions) + 内存反汇编⁴

jyy 用自制的 `addr-space` 工具展示了一个真实进程的地址空间:

- 4 个 **region** (内存区域)
- Region 1: **r-xp** (可读可执行, 私有) —— 代码段, 映射自 `a.out`
- 可以看到反汇编后的机器码和对应的指令
- 每个 **region** 都有明确的权限 (**r/w/x/p** 或 **s**)

地址空间的 region 结构

进程的地址空间不是一整块连续内存, 而是由多个**离散的 region** (区域) 组成。每个 region 有:

- 起始地址和长度
- 权限 (读/写/执行)
- 来源 (文件映射或匿名内存)

`/proc/[pid]/maps` 可以查看任何进程的 region 列表。

1.4 本章小结

地址空间 = 进程能访问的全部内存 = 一组带权限的 **region**。每个字节都有明确来源: ISA 决定结构、linker 决定布局、loader 初始化、运行时动态分配。用 `/proc/[pid]/maps` 和自制工具可以亲眼观察。

2 mmap: 内存映射

从“观察地址空间”到“操控地址空间”——**mmap** 是操作系统提供的最强大的内存管理 API。

⁴视频画面时间区间: 00:22:00-00:22:30。帧实际内容: VS Code 终端显示“Process 8544 Address Space (4 regions)”, 第一个 region `0x00000000400000-0x00000000404000` (16,384 bytes), Permission: `r-xp`, File mapping: `/tmp/vibe/addr-space/a.out`, 以及反汇编的前 1024 字节。

2.1 从 minimal.S 出发：地址空间里什么也没有

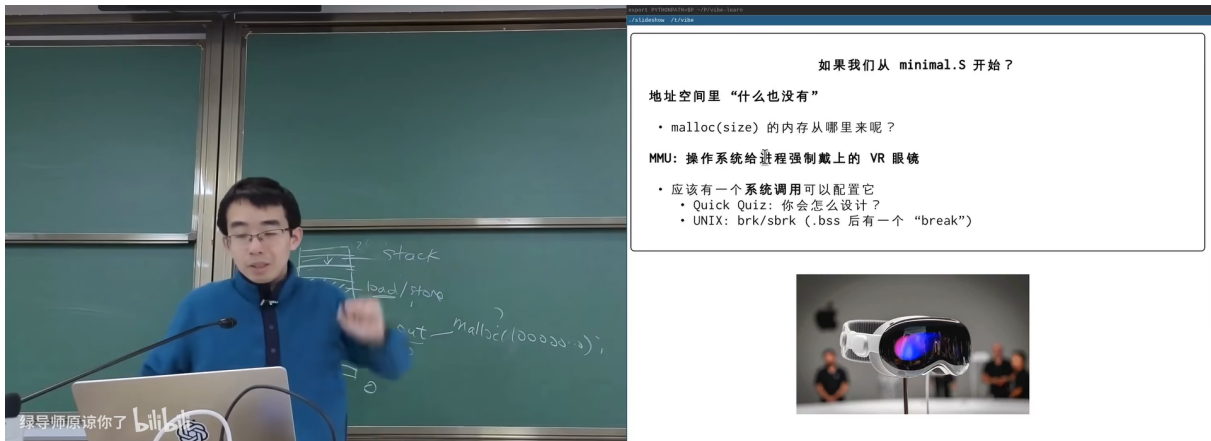


图 5: 如果从 minimal.S 开始？地址空间里“什么也没有”。MMU = OS 给进程戴上的 VR 眼镜⁵

MMU

MMU (Memory Management Unit) 让每个进程“看到”的地址空间和物理内存完全不同。类比：VR 眼镜让你看到虚拟世界——MMU 让进程看到虚拟地址空间。OS 控制这个“VR 眼镜”的内容，决定进程能看到什么、能访问什么。

malloc(size) 的内存从哪来？答案：进程向 OS 请求（通过 mmap 或 brk），OS 通过 MMU 把物理内存“映射”到进程的虚拟地址空间中。

2.2 mmap 的语义

mmap 系统调用

```
1 void *mmap(void *addr, size_t length, int prot, int flags,  
2           int fd, off_t offset);
```

在进程的地址空间中**创建**一个新的映射：

- **addr**: 期望的起始地址（通常传 NULL，让 OS 选择）
- **length**: 映射区域的大小
- **prot**: 权限（PROT_READ | PROT_WRITE | PROT_EXEC）
- **flags**: MAP_PRIVATE（私有，写时复制）/ MAP_SHARED（共享）/ MAP_ANONYMOUS（不关联文件）
- **fd, offset**: 文件映射时指定文件和偏移

对应的解除映射：munmap(addr, length)

修改权限：mprotect(addr, length, prot)

⁵视频画面时间区间：00:33:34–00:34:04。帧实际内容：slide “如果我们从 minimal.S 开始？地址空间里什么也没有”，提到 malloc(size) 的内存从哪来？MMU：操作系统给进程戴上的 VR 眼镜。附 Apple Vision Pro 图片。包含 Quick Quiz：brk 怎么设计？UNIX brk/sbrk：总有一个“break”指针。

2.3 strace 观察 mmap

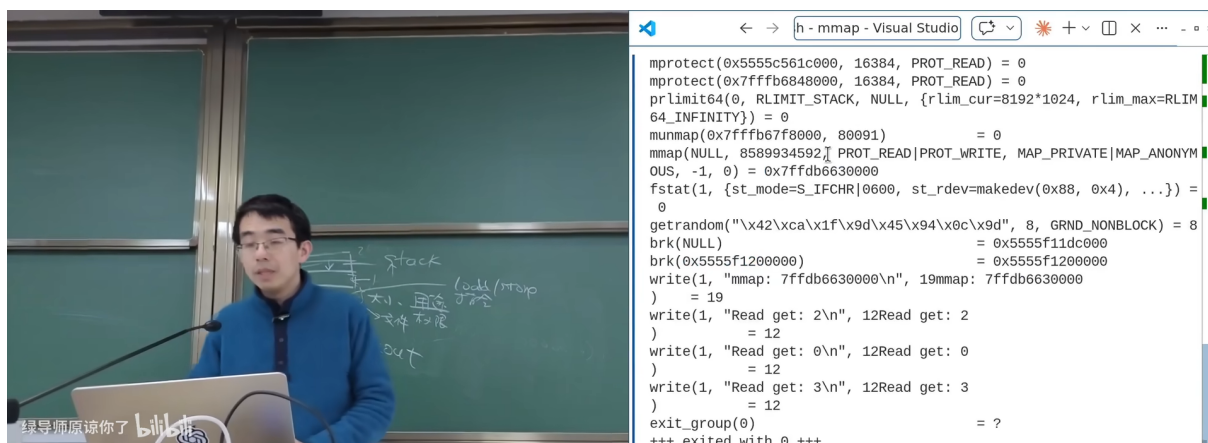


图 6: strace 观察 mmap 程序的系统调用序列⁶

jyy 用 strace 追踪了一个使用 mmap 的程序，可以看到完整的系统调用序列：

```
1 mprotect(0x5555c561c000, 16384, PROT_READ) = 0
2 mprotect(0x7fffb6648000, 16384, PROT_READ) = 0
3 munmap(0x7fffb67f8000, 80091) = 0
4 mmap(NULL, 8392832, PROT_READ|PROT_WRITE,
5     MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x...
6 write(1, "mmap: 7ffdb6630000\n", 19) = 19
7 +++ exited with 0 +++
```

Listing 2: strace 输出中的 mmap 相关调用

mmap 的两种典型用途

1. **文件映射**：把文件内容直接映射到内存，读写内存 = 读写文件。动态链接器 (ld.so) 就是用 mmap 把 .so 文件映射到进程地址空间。
2. **匿名映射** (MAP_ANONYMOUS)：不关联任何文件，纯粹分配内存。malloc 对大块内存的分配底层就是 mmap。

2.4 本章小结

mmap 是操作系统提供的“地址空间编辑器”——创建映射 (mmap)、删除映射 (munmap)、修改权限 (mprotect)。MMU 是实现虚拟地址空间的硬件基础。malloc 底层依赖 mmap/brk。

3 入侵进程的地址空间

这是本讲最精彩的部分——jyy 演示了如何“入侵”其他进程的地址空间，读写它们的内存。

⁶ 视频画面时间区间：00:41:04-00:41:34。帧实际内容：VS Code 终端显示 strace 输出，包含 mprotect、munmap、mmap(NULL, ..., PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS)、fstat、write 等系统调用序列。最后 exited with 0。

3.1 Hacking Address Spaces

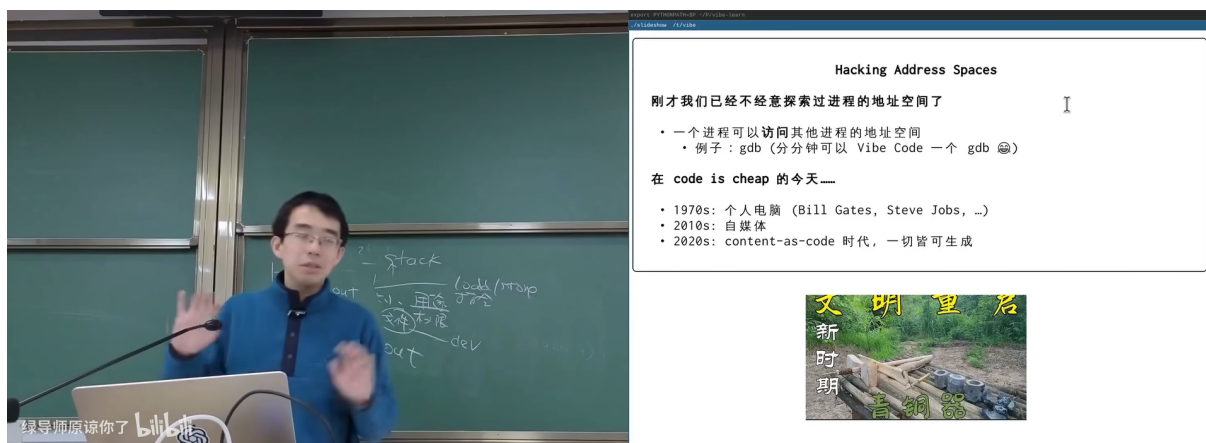


图 7: “入侵进程的地址空间”: 一个进程可以访问其他进程的地址空间⁷

jyy 揭示了一个令人兴奋的事实: 一个进程可以读写其他进程的地址空间。

入侵地址空间的接口

Linux 提供了多种方式让一个进程访问另一个进程的内存:

- `/proc/[pid]/mem`: 直接读写目标进程的虚拟内存
- `/proc/[pid]/maps`: 查看目标进程的内存布局
- `ptrace(PTRACE_PEEKDATA/POKEDATA, ...)`: 通过 `ptrace` 逐字读写
- `process_vm_readv` / `process_vm_writev`: 高效的跨进程内存读写

这就是 GDB 调试器的工作原理——它“入侵”被调试进程的地址空间, 读取变量、修改断点。

安全边界

能读写其他进程的内存意味着巨大的安全风险。Linux 通过以下机制限制:

- `ptrace` 需要 `CAP_SYS_PTRACE` 权限或父子关系
- `/proc/[pid]/mem` 受 `ptrace_scope` 控制
- SELinux / AppArmor 可以进一步限制

但在同一用户下, 这些限制通常是宽松的——这就是为什么“以 root 运行不安全的程序”如此危险。

⁷ 视频画面时间区间: 00:50:24–00:50:54。帧实际内容: slide “Hacking Address Spaces”——我们已经不经意间探索过进程的地址空间了; 一个进程可以读写其他进程的地址空间 (例如 `gdb-me` 用 Vibe Code 一个 `gdb` 插件)。下方列出 “code is cheap 的今天” 历史线: 1970s 个人电脑, 2010s 自媒体, 2020s content-as-code + 一切皆可编程。附文明建筑图片。

3.2 畅想的空间

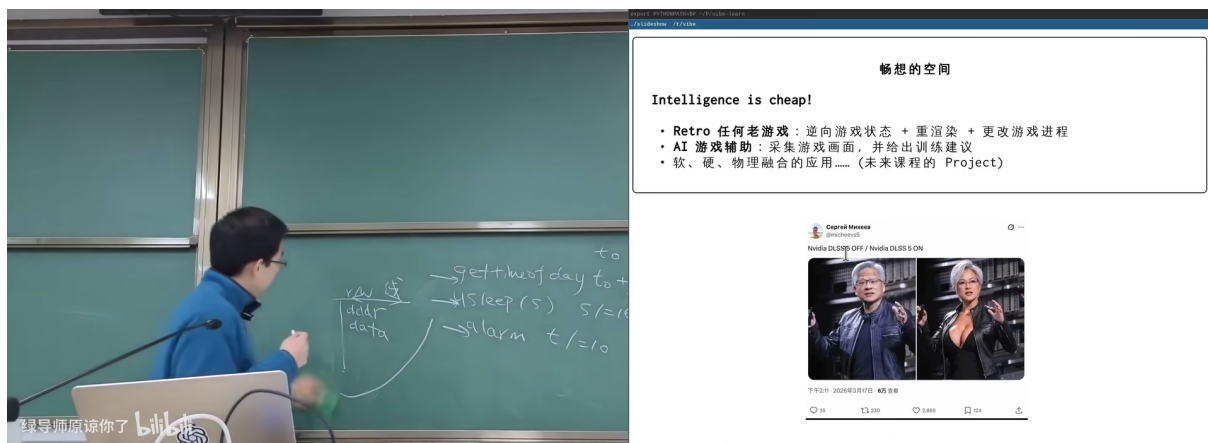


图 8: 畅想: Intelligence is cheap! Retro 在线密室、AI 游戏、物理融合⁸

jyy 在课程最后展开了一段精彩的畅想:

Intelligence is cheap!

当你能入侵任何进程的地址空间时, 结合 AI 的能力, 可以做什么?

- **Retro 在线密室逃脱**: 观察游戏进程状态 → 重回金 → 修改历史进程
- **AI 游戏精灵**: 采集游戏数据 → 训练 AI → 实时给出建议
- **物理融合的应用**: 软件侵入硬件控制..... (未来课程 Project)

“Code is cheap” 之后是 “Intelligence is cheap”——当 AI 能力足够便宜时, 操作系统级别的能力 (入侵地址空间) + AI 分析 = 无限可能。

3.3 本章小结

Linux 提供了完整的跨进程内存访问接口 (`/proc/[pid]/mem`、`ptrace`、`process_vm_readv`)。GDB 就是用这些接口实现的。结合 AI, 入侵地址空间的能力可以催生全新的应用形态。

4 总结与延伸

4.1 讲者的核心信息

地址空间 = region 集合 $\xrightarrow{\text{mmap 编辑}}$ 动态管理 $\xrightarrow{\text{/proc/mem}}$ 跨进程入侵

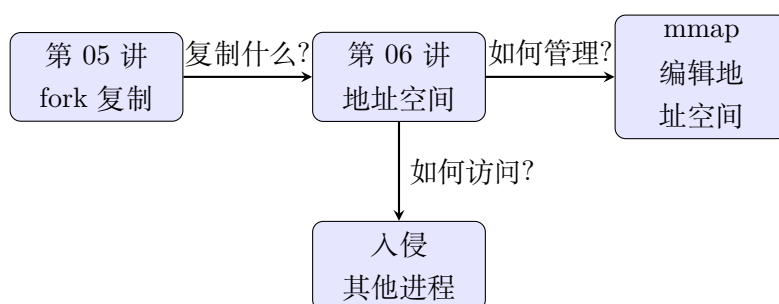
⁸视频画面时间区间: 01:09:09–01:09:39。帧实际内容: slide “畅想的空间”——Intelligence is cheap! 列出 Retro 在线密室逃脱 (进程观察 + 重回金 + 更改历史进程)、AI 游戏精灵 (采集数据训练、并行出训练建议)、物理融合的应用 (未来课程的 Project)。附黄仁勋演讲照片和 Claude/Anthropic 界面截图。

4.2 结构化提炼

四个核心 Takeaway

1. **地址空间 = 带权限的 region 集合**——不是连续内存，而是离散的映射区域。每个 region 有读/写/执行权限和来源（文件或匿名）。
2. **MMU = VR 眼镜**——OS 通过 MMU 让每个进程“看到”自己独有的虚拟地址空间，与物理内存完全解耦。这是进程隔离的硬件基础。
3. **mmap 是地址空间的编辑器**——创建映射(mmap)、删除(munmap)、改权限(mprotect)。malloc 对大块内存底层就是 mmap。
4. **入侵地址空间 = 调试/安全/AI 的基础**——/proc/[pid]/mem + ptrace 让你能读写任何进程的内存。GDB 用这个调试，恶意软件也用这个攻击。

4.3 跨章节关联



4.4 与前几讲的关联

- 第 05 讲 fork = 复制地址空间 → 第 06 讲解释了“地址空间”到底是什么（region 集合）
- 第 03 讲 CPU Reset 后内存全空 → 第 06 讲解释了内存中的每个字节是怎么来的（loader + mmap）
- 第 02 讲 strace 观测 syscall → 第 06 讲用 strace 观测 mmap/mprotect 调用

4.5 开放问题

1. MMU 的页表是怎么实现的？→ 后续的“虚拟内存实现”章节
2. Copy-on-Write 如何利用 mmap 和页表实现？
3. AI Agent 能否自动化入侵地址空间来调试程序？

4.6 拓展阅读

- man 2 mmap, man 2 mprotect, man 2 munmap
- man 5 proc (/proc 文件系统文档)
- OSTEP Chapter 13-15: Address Spaces, Memory API